

Exercice 1 : [1.5*5=7.5 points]

Donner les commandes nécessaires pour :

- a. Afficher les numéros des lignes vides d'un fichier `f.txt`

.....
.....

- b. Afficher les lignes qui se terminent par un point (".") du fichier `f.txt`

.....
.....

- c. Afficher les lignes qui commencent par + ou - et se terminent par ce même caractère trouvé au début du fichier `f.txt`.

.....
.....
.....

- d. Vérifier si un utilisateur passé en paramètres est connecté.

.....
.....
.....

- e. Trier numériquement la deuxième colonne d'un fichier `notes.txt`

.....
.....
.....

Exercice 2 : Le "make" [1+2+1+1+1+1=7 points]

Vous recevez les sources du logiciel "make". Vous copiez les fichiers dans un répertoire initialement vide, et vous imprimez le contenu de ce répertoire. On restera dans ce répertoire pendant tout l'exercice. Le Contenu du répertoire est :

```
-rw----- 1 u          2071 Jun 15 10:01 basepage.h
-rw----- 1 u          6841 Jun 15 10:01 check.c
-rw----- 1 u          3448 Jun 15 10:01 getenv.c
-rw----- 1 u          5416 Jun 15 10:01 getopt.c
-rw----- 1 u          8068 Jun 15 10:01 h.h
-rw----- 1 u        15843 Jun 15 10:01 input.c
-rw----- 1 u        10084 Jun 15 10:01 macro.c
-rw----- 1 u       25598 Jun 15 10:01 main.c
-rw----- 1 u       14933 Jun 15 10:01 make.c
-rw----- 1 u         4238 Jun 15 10:01 make.man
-rw----- 1 u         1176 Jun 15 10:01 Makefile
-rw----- 1 u       38345 Jun 15 10:01 makshell.c
-rw----- 1 u         5792 Jun 15 10:01 reader.c
-rw----- 1 u         3467 Jun 15 10:01 readme
-rw----- 1 u         8507 Jun 15 10:01 rules.c
-rw----- 1 u         1319 Jun 15 10:01 stat.h
-rw----- 1 u         6398 Jun 15 10:01 syserr.c
```

Constatant qu'il comporte un fichier de nom *Makefile*, dont le contenu est le suivant :

```
# Makefile for make utility.
CFLAGS =
LDFLAGS =
DESTDIR = /usr/bin
WORKDIR = /usr/local/sys/travail
BACKDIR = /usr/local/src/make
PROG = make
OBS = check.o input.o macro.o main.o make.o makshell.o\
      reader.o rules.o
LIBES = getenv.o getopt.o syserr.o
FILES = check.c input.c macro.c main.c make.c makshell.c\
      reader.c rules.c getenv.c getopt.c syserr.c\
      basepage.h h.h stat.h make.man makefile readme
$(PROG): $(OBS) $(LIBES)
    #Notez que ld : un éditeur de liens
    ld $(LDFLAGS) -o $(PROG) $(OBS) $(LIBES)
$(OBS): h.h
getenv.o main.o: basepage.h
main.o makshell.o: stat.h
install: $(PROG)
    cp $(PROG) $(DESTDIR)/$(PROG)
    rm $(PROG)
clean:
    rm $(OBS) $(LIBES) $(PROG)
erase: clean
    rm $(FILES)
backup:
    cp $(FILES) $(BACKDIR)
restore:
    cd $(BACKDIR)
    cp $(FILES) $(WORKDIR)
    cd $(WORKDIR)
help:
    @echo Makefile options:
    @echo help: Print this message
    @echo default: Create $(PROG)
    @echo install: Move $(PROG) to $(DESTDIR)
    @echo clean: Remove $(PROG) and all .o files
```

```
@echo erase: Erase ALL files
@echo backup: Copy source files to $(BACKDIR)
@echo restore: Copy files to $(WO(RKDIR)
```

1. Quelles sont les opérations faites par la cible install ?

2. A partir de ce *makefile*, déduire le graphe de dépendances entre les fichiers. Tous les fichiers potentiels doivent être mentionnés. Rappelez-vous que *make* utilise des règles implicites prédéfinies. Par exemple,

$$\% . O : \% . C$$

```
gcc -c $^ $@
```

pour construire les fichiers « .o » à partir de fichiers « .c » même si elles ces règles sont absentes du makefile.

3. Vous ne disposez pas encore du programme "make". Quelles sont les commandes shell nécessaires pour le créer (utiliser le Gnu Compiler Collection) ?

4. Les fichiers sont mis à jour, on modifie seulement le fichier "basepage.h", uniquement. Énoncez toutes les commandes qui sont exécutées si on lance la commande `make install`.

5. Vous créez le répertoire `"/usr/local/src/make"`, puis lancez la commande `"make backup"`. Que se passe-t-il?

6. Vous lancez la commande "make erase". Que se passe-t-il? Disposez-vous encore du programme "make"?

Exercice 3 (1+1+1.5+1+1 = 5.5 points)

On considère le système de gestion de fichiers de Unix. La taille d'un bloc est de 2 Ko (kilo-octets). Chaque pointeur (numéro de bloc) occupe 4 octets. Chaque inode comprend 10 liens directs, 1 lien indirect simple, 1 lien indirect double et 1 lien indirect triple.

1. Quel est le rôle de l'inode dans ce système ?

.....

.....

.....

.....

.....

2. Combien de blocs occupe un fichier vide ?

.....

.....

3. On considère un fichier de taille 500 000 Ko.

a. Combien de blocs de données sont nécessaires pour représenter ce fichier ?

.....

.....

.....

b. Combien d'adresses au maximum peut contenir un bloc d'index ?

.....

.....

c. Combien de blocs d'index sont nécessaires pour représenter ce fichier ?

.....

.....

.....

.....

4. Quelle est la taille minimale que doit avoir un fichier pour qu'on soit obligé d'utiliser le lien indirect triple ? Justifiez.

.....

.....

5. Quelle est la taille maximale d'un fichier qu'on peut représenter avec ce système ? Quel serait le nombre de blocs de données et de blocs d'index ? Justifiez.

.....

.....

.....

.....